

MTH208: Worksheet 15

C++ and R via Rcpp

By this time, you know that R (like many high-level languages) can be slower than expected. A lot of other high-level languages have this problem.

To make sure we can use R efficiently, we have a package called `Rcpp` that allows us to use C++ from within R.

Let us see this by an example of adding two numbers:

```
# Consider the following R function
addR <- function(x, y)
{
  return(x + y)
}

library(Rcpp)
# cppFunction() is a function that takes a bunch of C++ codes (all in quotes)
# and then saves and compiles it.

# In C++, we have to declare the type of every object.
# The function we are making is addC() which accepts two integers
# and returns the sum of the two integers.
cppFunction('int addC(int x, int y) {
  int sum = x + y;
  return sum;
}')
```

When you run the above lines it takes some time for R to compile the function and get it ready. Once it has finished compiling, it makes available a function `addC` for you to use in R, that is actually written in C++.

```
addR(3, 4)
```

```
[1] 7
```

```
addC(3, 4)
```

[1] 7

A typically C++ code has the following form:

```
# for Rcpp usage put
cppFunction('
output_type name_of_func(input_type input1, input_type input2, ...){

    line1 ... ;
    line2 ... ;
    .
    .
    return output;
}
')
```

Using Rcpp, we can also create functions that take vector inputs. Let's create functions that calculate the Euclidean distance between two vectors in both R and Rcpp.

```
# In R
EucR <- function(x, y)
{
    rtn <- sqrt(sum( (x-y)^2 ))
    return(rtn)
}

# In C++ using Rcpp

cppFunction('double EucC(NumericVector x, NumericVector y) {
    double track = 0;
    int n = x.size();

    for(int i = 0; i < n; i++){
        track = track + pow( (x[i] - y[i]), 2);
    }
    track = sqrt(track);
    return track;
}
')
```

Please notice the change in syntax for loops etc. This is because inside `cppFunction` we are using C++ compatible code. Run the following and make sure the output is TRUE:

```
x <- 1:10
y <- 3:12
```

```
# all.equal checks whether result is the same
all.equal(EucR(x, y), EucC(x, y))
```

To find out different kinds of data types in Rcpp, see this link here https://teuder.github.io/rcpp4everyone_en/070_data_types.html.

Problems

1. In the previous example, define x and y to be arbitrary vectors of length 10^4 . Benchmark the performance of `EucR` and `EucC`.
2. Consider a vector $x = (x_1, \dots, x_n)$, and suppose we want to calculate:

$$\frac{\log(x_i)}{\sum_{k=1}^n \log(x_k)}$$

The function `func` below calculates the above for a given vector `vec`.

```
func <- function(vec)
{
  n <- length(vec)

  # for tracking sum and log
  sum.log <- 0
  log.of.vec <- numeric(length(n))

  # calculating logs and sum for each element
  for(i in 1:n)
  {
    log.of.vec[i] <- log(vec[i])
    sum.log <- sum.log + log.of.vec[i]
  }

  # fraction
  frac <- log.of.vec/sum.log
  return(frac)
}
```

- i. Rewrite the function in R in the most efficient way possible. Call this function `funcR`.
 - ii. Rewrite the function using Rcpp and call the function `funcC`. Benchmark `func`, `funcR`, and `funcC`. Which is fastest?
3. Write an Rcpp function that adds two matrices together.

4. Writes an Rcpp equivalent of the function `colSums` and benchmark it against `colSums` for a large vector input.
5. Write an Rcpp function that takes a vector and returns a logical vector of the same length, indicating whether the value is positive (TRUE) or not (FALSE).
6. Redo Problem 4 in Worksheet 14 in Rcpp and benchmark with the other two implementations.